Unidad 2

Estructuras de control

Selección con if y switch, repetición con while, do…while y for, y salto con break, continue y etiquetas.

Sin estructuras de control, un programa ejecuta sus instrucciones de arriba abajo, una vez cada una. Las estructuras de control rompen esa secuencia: permiten elegir un camino u otro, repetir un bloque, o saltar fuera de él.

Estructuras selectivas

if / else if / else

Ejecuta un bloque u otro según se cumplan unas condiciones. Se evalúan en orden y se ejecuta sólo el primer bloque cuya condición sea cierta:

```
const temperatura = 23;

if (temperatura < 10) {
  console.log('Frío');
} else if (temperatura < 25) {
  console.log('Templado');
} else {
  console.log('Calor');
}
// → Templado
```

Las llaves son opcionales cuando el bloque tiene una sola instrucción, pero ponerlas siempre evita errores al añadir líneas más tarde.

switch

Cuando la decisión depende del valor de **una única expresión** y hay varias opciones, `switch` se lee mejor que una cadena de `else if`:

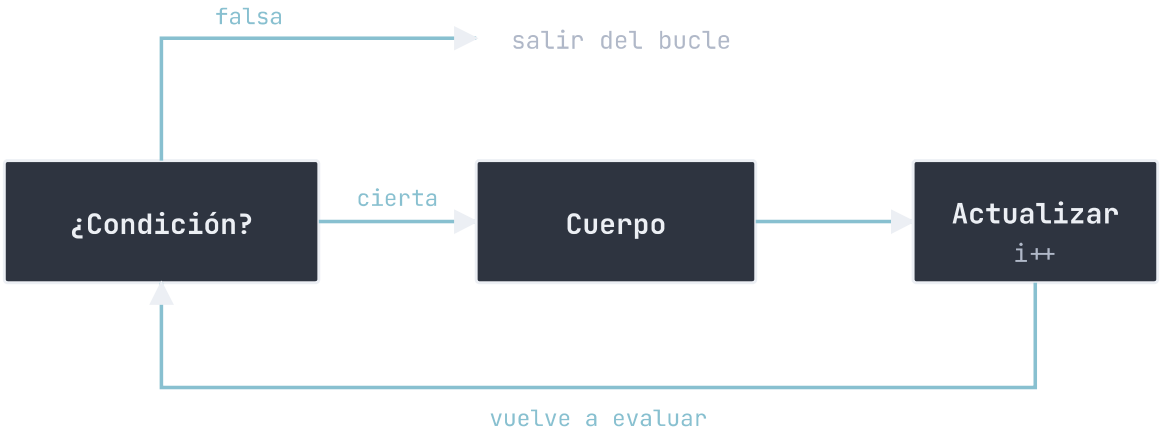
```
const dia = 'martes';

switch (dia) {
  case 'sábado':
  case 'domingo':
    console.log('Fin de semana');
    break;
  case 'viernes':
    console.log('Casi');
    break;
  default:
    console.log('Entre semana');
}
```

Dos detalles: la comparación es estricta (`===`), y sin `break` la ejecución **continúa** en el siguiente `case` . Ese comportamiento se usa a propósito para agrupar casos, como `'sábado'` y `'domingo'` arriba, pero olvidado es una fuente clásica de errores. `default` recoge lo que no encaja en ningún `case` .

Estructuras repetitivas

Un bucle repite un bloque mientras se cumple una condición. El diagrama muestra el ciclo que siguen los tres tipos:



Mientras la condición sea cierta se repite cuerpo y actualización. La salida sólo ocurre cuando la condición pasa a ser falsa: si nunca lo hace, el bucle es infinito.

while

Evalúa la condición **antes** de cada vuelta. Si es falsa desde el principio, el cuerpo no se ejecuta ninguna vez:

```
let intentos = 0;

while (intentos < 3) {
  console.log('Intento', intentos + 1);
  intentos++;
}
```

do … while

Evalúa la condición **después** de cada vuelta, así que el cuerpo se ejecuta al menos una vez. Es la elección natural cuando hay que hacer algo antes de poder decidir si repetir, como pedir un dato y validarlo:

```
let numero;

do {
  numero = Math.floor(Math.random() * 10);
  console.log('Ha salido', numero);
} while (numero !== 7);
```

for

Agrupa en la cabecera las tres piezas del diagrama: inicialización, condición y actualización. Es el bucle habitual cuando se sabe cuántas vueltas hay que dar o se recorre algo con índice:

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
// → 1 2 3 4 5
```

La variable `i` se declara con `let` dentro del `for` y sólo existe dentro del bucle. El error más común es confundir `<` con `<=` y dar una vuelta de más o de menos: comprueba siempre el primer y el último valor.

> ▶ Cómo evitar un bucle infinito

Estructuras de salto

Alteran el flujo normal de un bucle desde dentro.

break

Termina el bucle más cercano de golpe, sin importar la condición:

```
for (let i = 1; i <= 10; i++) {
  if (i * i > 30) {
    break;      // i = 6: 36 > 30, se sale
  }
  console.log(i);
}
// → 1 2 3 4 5
```

continue

Salta el resto de la vuelta actual y pasa a la siguiente (en un `for`, ejecutando antes la actualización):

```
for (let i = 1; i <= 6; i++) {
  if (i % 2 === 0) {
    continue;   // los pares se saltan
  }
  console.log(i);
}
// → 1 3 5
```

Etiquetas

Con bucles anidados, `break` y `continue` afectan sólo al más interno. Una etiqueta (un nombre seguido de dos puntos, delante del bucle) permite referirse a uno exterior:

```
filas: for (let f = 0; f < 3; f++) {
  for (let c = 0; c < 3; c++) {
    if (c === 2) {
```

```
      // pasa a la siguiente fila, saltando lo que quede de esta
      continue filas;
    }
    console.log(f, c);
  }
}
// → 0 0, 0 1, 1 0, 1 1, 2 0, 2 1
```

Se usan poco; si te ves necesitando etiquetas, suele ser señal de que el bucle interior debería ser una función aparte.

Ejemplo 1: if / else if / else

Escribe la función calificacion(nota) que devuelva Insuficiente si la nota es menor que 5, Suficiente de 5 a menos de 6, Bien de 6 a menos de 7, Notable de 7 a menos de 9 y Sobresaliente de 9 en adelante.

```
function calificacion(nota) {

}

console.log(calificacion(4));
console.log(calificacion(8.5));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 1

Ejemplo 2: bucle for

Escribe la función sumaHasta(n) que devuelva la suma de todos los enteros desde 1 hasta n, ambos incluidos, usando un bucle for. Para n = 4 el resultado es 1 + 2 + 3 + 4 = 10.

```
function sumaHasta(n) {
  let suma = 0;

  return suma;
}

console.log(sumaHasta(4));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 2

Ejemplo 3: salir del bucle con break

Escribe la función primerMultiplo(lista, k) que recorra el array lista y devuelva el primer elemento que sea múltiplo de k. Si no hay ninguno, devuelve null. Usa break para dejar de buscar en cuanto lo encuentres.

```
function primerMultiplo(lista, k) {
  let encontrado = null;

  return encontrado;
}

console.log(primerMultiplo([7, 9, 12, 15, 18], 6));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 3

De este módulo

T2.3 Ejercicios: estructuras de control
Ocho programas con if, switch y bucles, con validación del formato de entrada.
